Puppet Expert Seminar

Day 3

Ruby, Custom Extensions, Security, Performance, Foreman, and Best Practices

Day 3 — Agenda

Part 14 — Ruby for Puppet Developers

- Ruby fundamentals for Puppet context
- Where Ruby appears in the Puppet ecosystem

Part 15 — Puppet Functions in Ruby

- Function API, dispatch, typed parameters
- Testing custom functions

Part 16 — Custom Facts with Facter

- Ruby and external facts, structured facts
- Debugging and security implications

Part 17 — Custom Types and Providers

- Type definition, properties, parameters
- Provider methods: exists?, create, destroy, flush
- Testing idempotency

Part 18 — Puppet Tasks, Plans, and Bolt

- Task and plan concepts
- Bolt architecture and inventory

Part 19 — Security in Puppet Environments

- Certificate trust model
- EYAML key management, secret rotation
- Code security and supply chain risks

Part 20 — Performance Optimization and Scaling

- Identifying bottlenecks: server, code, agent, PuppetDB
- JRuby tuning, code performance patterns

Part 21 — Foreman as Lifecycle Management and ENC

- Foreman architecture, Smart Proxy
- ENC classification, host groups, Smart Class Parameters

Part 22 — Enterprise Puppet Best Practices

Part 14 — Ruby for Puppet Developers

Ruby Fundamentals — Variables and Data Structures

```ruby
# Strings
name = "puppet"
greeting = "Hello, #{name.capitalize}!"   # string interpolation
multiline = <<~HEREDOC
  This is a
  multiline string
HEREDOC

# Arrays
packages = ['nginx', 'curl', 'vim']
packages.push('htop')
packages.each { |pkg| puts pkg }
first = packages.first
last  = packages.last
subset = packages[0..1]    # ['nginx', 'curl']

# Hashes
config = {
  host: 'localhost',       # symbol key (preferred in Ruby)
  port: 5432,
  ssl:  true,
}
config[:host]              # => 'localhost'
config.key?(:timeout)      # => false
config.fetch(:port, 3306)  # => 5432 (or default if missing)
```

Ruby Methods, Blocks, and Lambdas

```ruby
# Method definition
def normalize_hostname(hostname)
  hostname.downcase.strip.gsub(/[^a-z0-9\-.]/, '-')
end

# Methods with optional parameters and defaults
def create_user(name, uid:, shell: '/bin/bash', groups: [])
  { name: name, uid: uid, shell: shell, groups: groups }
end

# Blocks — passed to methods that yield
[1, 2, 3].map { |n| n * 2 }       # => [2, 4, 6]  (single line)

[1, 2, 3].select do |n|            # multi-line block
  n.odd?
end
# => [1, 3]

# Reduce / inject
total = [10, 20, 30].reduce(0) { |sum, n| sum + n }   # => 60
```

Ruby Classes and Modules

```ruby
# Class definition
class PuppetNode
  attr_reader :certname, :environment

  def initialize(certname, environment: 'production')
    @certname    = certname
    @environment = environment
    @facts       = {}
  end

  def add_fact(key, value)
    @facts[key] = value
    self     # return self for method chaining
  end

  def to_hash
    { certname: @certname, environment: @environment, facts: @facts }
  end

  def to_s
    "PuppetNode(#{@certname}, #{@environment})"
  end
end

node = PuppetNode.new('web01.example.com', environment: 'staging')
node.add_fact('os_family', 'RedHat').add_fact('datacenter', 'ams01')
puts node.certname        # => web01.example.com
```

Ruby for Puppet — Where It Appears

Location	Ruby role
`lib/puppet/functions/mymodule/fname.rb`	Custom Puppet function (modern API)
`lib/facter/myfact.rb`	Custom Ruby fact
`lib/puppet/type/mytype.rb`	Custom resource type definition
`lib/puppet/provider/mytype/linux.rb`	Custom resource provider

Location	Ruby role
`spec/classes/init_spec.rb`	rspec-puppet test (Ruby test code)
`spec/spec_helper.rb`	Test configuration (Ruby)
`Gemfile`	Gem dependency management
`Rakefile`	Task runner (rake test, rake lint)
`puppetserver` JVM internals	Puppet Server itself runs on JRuby
`facter` binary	Facter is a Ruby gem

> You do not need to be a Ruby expert to work with Puppet — but reading and writing basic Ruby is essential for custom extensions and testing.

Ruby Error Handling

```ruby
# begin/rescue/ensure pattern
def read_config_file(path)
  begin
    content = File.read(path)
    JSON.parse(content)
  rescue Errno::ENOENT => e
    Puppet.warning "Config file not found: #{path} — using defaults"
    {}
  rescue JSON::ParserError => e
    Puppet.err "Invalid JSON in #{path}: #{e.message}"
    raise Puppet::Error, "Cannot parse config file: #{path}"
  ensure
    Puppet.debug "Finished reading #{path}"  # always runs
  end
end

# Puppet-specific error classes
raise Puppet::Error, "Resource validation failed: port must be 1-65535"
raise Puppet::ParseError, "Invalid manifest syntax"

# Logging in Ruby Puppet code
Puppet.debug   "Detailed debugging info — only shown with --debug"
Puppet.info    "Normal informational message"
Puppet.warning "Something unexpected but non-fatal"
Puppet.err     "Error that will be recorded in the report"
```

Part 15 — Puppet Functions in Ruby

Modern Function API

```ruby
# lib/puppet/functions/mymodule/validate_port.rb

Puppet::Functions.create_function(:'mymodule::validate_port') do

  # dispatch: maps a signature to a method
  dispatch :validate_integer do
    param 'Integer', :port
    return_type 'Boolean'
  end

  dispatch :validate_string do
    param 'String', :port
    return_type 'Boolean'
  end

  def validate_integer(port)
    port >= 1 && port <= 65535
  end

  def validate_string(port)
    begin
      int_port = Integer(port)
      validate_integer(int_port)
    rescue ArgumentError
      false
    end
  end
end
```

```
# Usage in a manifest
if mymodule::validate_port($listen_port) {
  # ...
} else {
  fail("Invalid port: ${listen_port}")
}
```

Functions — Accessing Scope, Facts, and Calling Other Functions

```
Puppet::Functions.create_function(:'mymodule::node_class') do

  dispatch :resolve do
    param 'String', :role
    return_type 'String'
  end

  def resolve(role)
    # Call another Puppet function
    env = call_function('lookup', 'environment_name', { 'default_value' => 'production' })

    # Access facts through the scope (available from the closure_scope)
    facts = closure_scope['facts']
    os_family = facts['os']['family']

    "role::#{role}_#{os_family.downcase}_#{env}"
  end
end
```

Functions — Returning Complex Data

```ruby
# lib/puppet/functions/mymodule/build_user_hash.rb

Puppet::Functions.create_function(:'mymodule::build_user_hash') do

  dispatch :build do
    param 'Array[Struct[{name => String, uid => Integer, Optional[shell] => String}]]', :users
    return_type 'Hash[String, Hash]'
  end

  def build(users)
    result = {}
    users.each do |user|
      result[user['name']] = {
        'uid'   => user['uid'],
        'shell' => user.fetch('shell', '/bin/bash'),
        'home'  => "/home/#{user['name']}",
      }
    end
    result
  end
end
```

Testing Custom Functions

```ruby
# spec/functions/validate_port_spec.rb
require 'spec_helper'

describe 'mymodule::validate_port' do
  it { is_expected.not_to be_nil }

  context 'with valid integer port' do
    it { is_expected.to run.with_params(80).and_return(true) }
    it { is_expected.to run.with_params(443).and_return(true) }
    it { is_expected.to run.with_params(65535).and_return(true) }
    it { is_expected.to run.with_params(1).and_return(true) }
  end

  context 'with invalid integer port' do
    it { is_expected.to run.with_params(0).and_return(false) }
    it { is_expected.to run.with_params(65536).and_return(false) }
    it { is_expected.to run.with_params(-1).and_return(false) }
  end

  context 'with string port' do
    it { is_expected.to run.with_params('8080').and_return(true) }
    it { is_expected.to run.with_params('notaport').and_return(false) }
  end

  context 'with wrong type' do
    it { is_expected.to run.with_params(['array']).and_raise_error(ArgumentError) }
  end
end
```

Part 16 — Custom Facts with Facter

Custom Ruby Facts

```ruby
# lib/facter/app_version.rb

Facter.add('myapp_version') do
  confine :kernel => 'Linux'          # only run on Linux
  confine { File.exist?('/usr/bin/myapp') }   # only if binary exists

  setcode do
    version_output = Facter::Core::Execution.execute(
      '/usr/bin/myapp --version 2>/dev/null',
      on_fail: :raise
    )
    # Parse "myapp version 2.3.1" → "2.3.1"
    version_output.match(/version\s+(\S+)/)&.captures&.first
  end
end
```

```ruby
# lib/facter/datacenter_info.rb — structured fact

Facter.add('datacenter') do
  setcode do
    result = {}

    # Read from cloud metadata or local config file
    if File.exist?('/etc/datacenter.conf')
      File.read('/etc/datacenter.conf').each_line do |line|
        key, val = line.chomp.split('=', 2)
        result[key.strip] = val&.strip if key && val
      end
    end

    result   # returns a hash → structured fact
  end
end
```

External Facts

```bash
#!/bin/bash
# facts.d/datacenter.sh — executable external fact
# Must output key=value pairs, one per line

echo "datacenter=ams01"
echo "rack=rack-42"
echo "pod=pod-a"
```

```yaml
# facts.d/hardware_info.yaml — YAML external fact (static)
---
hw_vendor: "Dell"
hw_model: "PowerEdge R740"
hw_support_tier: "gold"
```

```python
# External facts can also return structured data (JSON):
# facts.d/cloud_metadata.py
#!/usr/bin/env python3
import json, urllib.request
try:
    r = urllib.request.urlopen('http://169.254.169.254/latest/dynamic/instance-identity/document', timeout=1)
    data = json.loads(r.read())
    print(json.dumps({'cloud': {'provider': 'aws', 'region': data['region'],
                                'instance_id': data['instanceId']}}))
except Exception:
    print(json.dumps({'cloud': {'provider': 'unknown'}}))
```

Fact Debugging

```
# View all facts on the agent (including custom)
/opt/puppetlabs/bin/facter -p   # -p includes Puppet-specific facts

# Debug a specific custom fact
FACTER_TRACE=1 /opt/puppetlabs/bin/facter myapp_version

# Check if custom fact is being found
/opt/puppetlabs/bin/facter --custom-dir /etc/puppetlabs/code/environments/production/modules/mymodule/lib/facter

# On Puppet Server — compile catalog to see fact values
puppet catalog compile agent01.example.com --render-as json \
  | python3 -m json.tool | grep -A5 '"parameters"'

# Common problems:
#   Fact file has Ruby syntax error → check with: ruby -c lib/facter/myfact.rb
#   Fact not in /opt/puppetlabs/facter/facts.d/ → pluginsync not working
#   Confine rejects the platform → add Puppet.debug to check
```

Security Implications of Facts

Critical security rule: Facts are submitted by the agent and **must not be trusted for security decisions**.

```
# UNSAFE — agent could fake this fact
if $facts['is_privileged_node'] {
  include profile::sensitive_data
}

# SAFE — trusted facts come from the signed certificate (cannot be faked)
if $trusted['extensions']['1.3.6.1.4.1.34380.1.1.13'] == 'privileged' {
  include profile::sensitive_data
}
```

Trusted fact extensions are set at certificate signing time using policy-based autosigning:

```
# /etc/puppetlabs/puppet/autosign.rb
# Sets OID extensions from CSR challenge password
Puppet::SSL::CertificateAuthority::CertificateSigningPolicy.new do |policy|
  policy.allow_extensions = ['pp_role', 'pp_environment', 'pp_datacenter']
end
```

> Use `$trusted` for security-relevant decisions. Use `$facts` for system capability decisions.

Part 17 — Developing Custom Types and Providers

When to Build a Custom Type

```
Decision tree:
  Can I manage this with existing resource types (file, exec, package)?
  → Yes: use them. Don't build a custom type.

  Can I wrap it cleanly in a defined type?
  → Yes: use a defined type. Simpler, testable with rspec-puppet.

  Is the resource stateful? Does it have properties that Puppet
  should read, compare, and apply idempotently?
  → Does it need prefetch (bulk read for performance)?
  → Does it have complex state management?
  → Is the provider OS-specific?
  → Yes to any: build a custom type + provider.
```

Good candidates for custom types:

- Database users and permissions (complex state, multiple attributes)
- Firewall rules with complex matching

Custom Type Definition

```ruby
# lib/puppet/type/myapp_user.rb

Puppet::Type.newtype(:myapp_user) do
  @doc = "Manages users in MyApp's internal user database."

  ensurable   # adds ensure => present/absent automatically

  # Namevar — the primary key / title
  newparam(:name, namevar: true) do
    desc 'The username. Must be lowercase alphanumeric.'
    validate do |value|
      raise ArgumentError, "Invalid username: #{value}" unless value.match?(/\A[a-z0-9_]+\z/)
    end
  end

  # Properties — Puppet reads current state and compares to desired
  newproperty(:email) do
    desc 'User email address.'
    validate do |value|
      raise ArgumentError, "Invalid email: #{value}" unless value.include?('@')
    end
  end

  newproperty(:role) do
    desc 'User role. One of: admin, editor, viewer.'
    newvalues(:admin, :editor, :viewer)
  end

  # Parameter — influences behaviour but is not read back from the system
  newparam(:force_password_reset) do
    desc 'When true, force password reset on next login.'
    newvalues(:true, :false)
    defaultto :false
  end
end
```

Type Properties vs. Parameters

Understanding the difference is critical for correct idempotency behaviour:

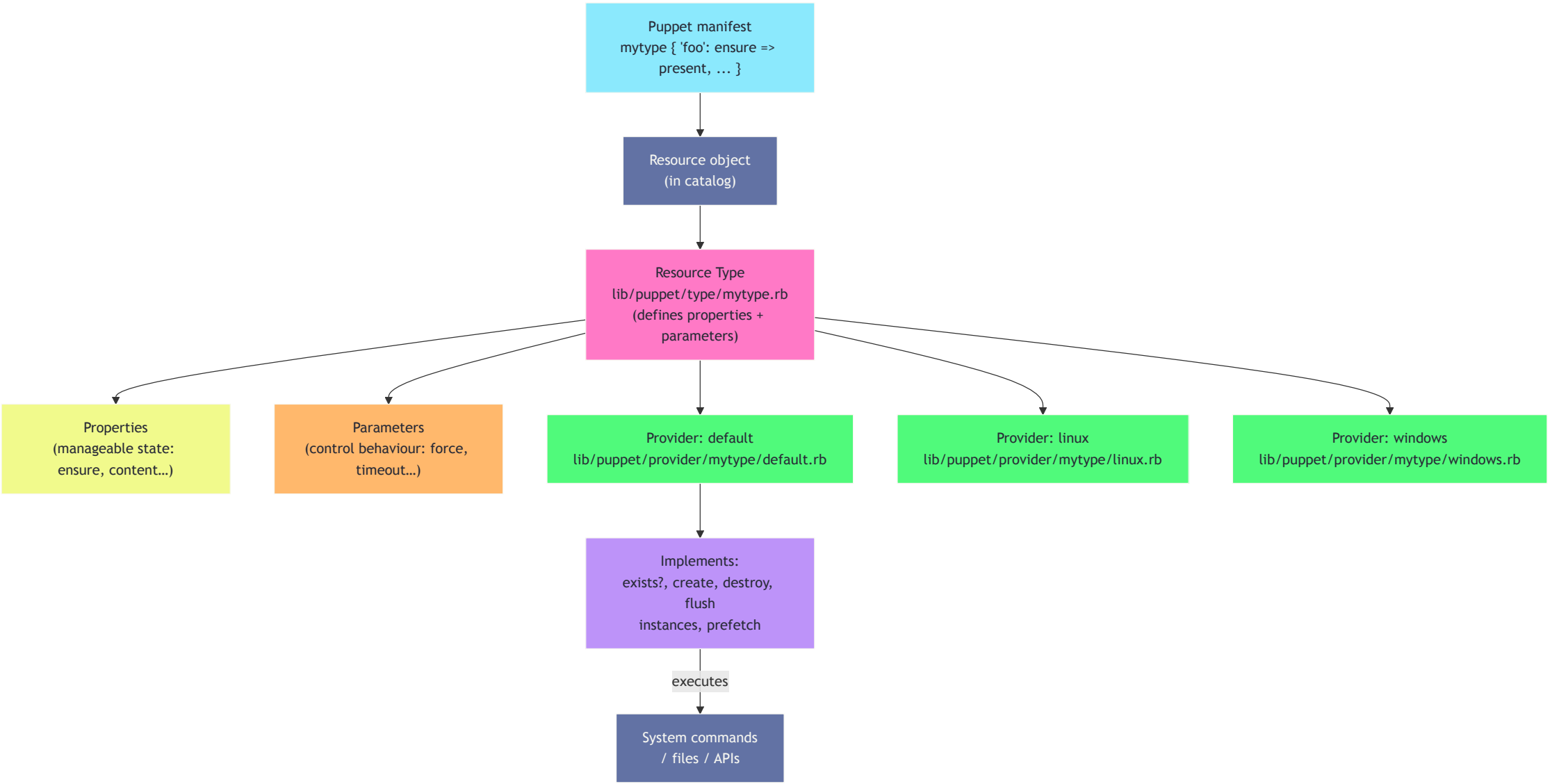
Concept	Declaration	Puppet behaviour	Example
Property	`newproperty(:email)`	Puppet reads current value via `retrieve`, compares, syncs if different	`email`, `password_hash`, `quota`
Parameter	`newparam(:name)`	Used as input to the provider; Puppet never reads it back	`name` (namevar), `force`, `provider`
Ensurable	`ensurable`	Adds `ensure => present/absent`; calls `exists?`, `create`	Standard — almost always include

```ruby
Puppet::Type.newtype(:myapp_user) do
  ensurable   # adds ensure property — calls exists?, create, destroy

  newparam(:name, namevar: true) do
    desc 'Username — used for lookup, never synced by Puppet'
  end

  newproperty(:email) do
    desc 'Email address — Puppet reads current value and syncs if different'
    validate { |v| raise ArgumentError, 'invalid email' unless v.include?('@') }
  end

  newproperty(:role) do
    desc 'User role in the application'
    newvalues(:admin, :editor, :viewer)
    defaultto :viewer
  end

  # Provider feature — mark capabilities the provider may or may not support
  feature :manages_quota, 'The provider can set storage quotas'
  newproperty(:quota_mb, required_features: :manages_quota) do
    desc 'Storage quota in megabytes'
    validate { |v| raise ArgumentError unless Integer(v) > 0 }
    munge { |v| Integer(v) }   # normalise to Integer regardless of input type
  end
end
```

Custom Provider Implementation

```ruby
# lib/puppet/provider/myapp_user/rest.rb

Puppet::Type.type(:myapp_user).provide(:rest) do
  desc "Manages MyApp users via the REST API."

  confine    :feature => :myapp_api   # only suitable if feature exists
  defaultfor :kernel => 'Linux'

  # Bulk-read all users at once (called once per catalog run — performance!)
  def self.prefetch(resources)
    instances.each do |provider|
      if (resource = resources[provider.name])
        resource.provider = provider
      end
    end
  end

  # Return a provider instance for every existing user
  def self.instances
    api_get('/api/users').map do |user_data|
      new(
        name:   user_data['username'],
        email:  user_data['email'],
        role:   user_data['role'].to_sym,
        ensure: :present,
      )
    end
  end

  # Does this resource exist?
  def exists?
    @property_hash[:ensure] == :present
  end

  # Create the resource
  def create
    api_post('/api/users', {
      username: resource[:name],
      email:    resource[:email],
      role:     resource[:role].to_s,
    })
    @property_hash[:ensure] = :present
  end

  # Destroy the resource
  def destroy
    api_delete("/api/users/#{resource[:name]}")
    @property_hash[:ensure] = :absent
  end

  # Flush staged property changes (called after create/all property setters)
  def flush
    return if @property_hash.empty?
    api_put("/api/users/#{resource[:name]}", {
      email: resource[:email],
      role:  resource[:role].to_s,
    })  if @property_hash[:ensure] == :present
  end

  # Property getters and setters (auto-generated by prefetch into @property_hash)
  def email;        @property_hash[:email];        end
  def email=(val);  @property_hash[:email] = val;  end
  def role;         @property_hash[:role];         end
  def role=(val);   @property_hash[:role]  = val;  end

  private

  def api_base
    resource.catalog.resource('Class[myapp::api]')&.parameters[:base_url] ||
      'http://localhost:9090'
  end

  def api_get(path)
    uri = URI("#{api_base}#{path}")
    response = Net::HTTP.get_response(uri)
    JSON.parse(response.body)
  end

  def api_post(path, body)
    uri = URI("#{api_base}#{path}")
    http = Net::HTTP.new(uri.host, uri.port)
    req  = Net::HTTP::Post.new(uri.path, 'Content-Type' => 'application/json')
    req.body = body.to_json
    http.request(req)
  end

  def api_put(path, body)
    uri = URI("#{api_base}#{path}")
    http = Net::HTTP.new(uri.host, uri.port)
    req  = Net::HTTP::Put.new(uri.path, 'Content-Type' => 'application/json')
    req.body = body.to_json
    http.request(req)
  end

  def api_delete(path)
    uri = URI("#{api_base}#{path}")
    http = Net::HTTP.new(uri.host, uri.port)
    http.request(Net::HTTP::Delete.new(uri.path))
  end
end
```

Custom Type and Provider Architecture



Testing Custom Types and Providers

```ruby
# spec/unit/puppet/type/myapp_user_spec.rb
require 'spec_helper'

describe Puppet::Type.type(:myapp_user) do
  let(:resource) do
    described_class.new(name: 'alice', email: 'alice@example.com', role: :editor)
  end

  it 'creates a resource with required attributes' do
    expect(resource[:name]).to eq('alice')
    expect(resource[:email]).to eq('alice@example.com')
    expect(resource[:role]).to eq(:editor)
  end

  it 'rejects invalid username' do
    expect {
      described_class.new(name: 'Alice With Spaces')
    }.to raise_error(Puppet::ResourceError, /Invalid username/)
  end

  it 'rejects invalid email' do
    expect {
      described_class.new(name: 'bob', email: 'not-an-email')
    }.to raise_error(Puppet::ResourceError, /Invalid email/)
  end

  it 'accepts valid roles' do
    [:admin, :editor, :viewer].each do |role|
      expect {
        described_class.new(name: 'alice', role: role)
      }.not_to raise_error
    end
  end

  it 'rejects invalid roles' do
    expect {
      described_class.new(name: 'alice', role: :superuser)
    }.to raise_error(Puppet::ResourceError)
  end
end
```

Part 18 — Puppet Tasks, Plans, and Bolt

Puppet Tasks vs. Configuration Management

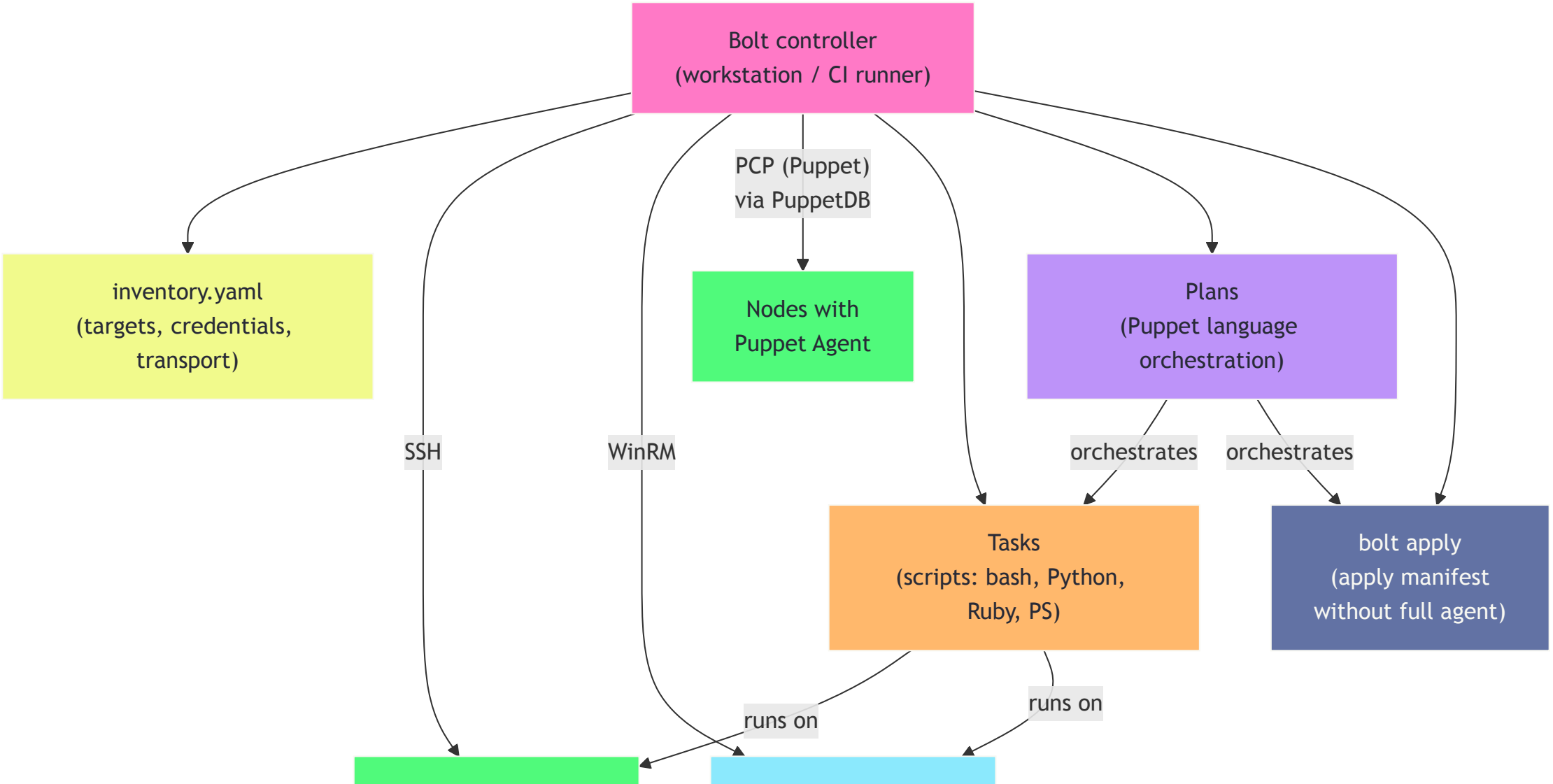
	Puppet agent run	Puppet Task (Bolt)
Model	Desired state / convergence	Imperative / procedural
Trigger	Scheduled (every 30 min)	On-demand
Persistence	Always enforced	Run once
Use case	Long-lived config state	Ad-hoc operations
Example	Ensure nginx is installed	Restart a service NOW
Agent required	Yes	No (for Bolt via SSH)

> Tasks are not a replacement for configuration management — they are a complement for operational tasks that are not configuration state.

Writing a Puppet Task

```
# tasks/restart_service.sh
#!/bin/bash
# @summary Restart a system service and verify it is running.

set -e

SERVICE="${PT_service}"   # PT_ prefix = task parameter

if [ -z "$SERVICE" ]; then
  echo '{"status": "failure", "message": "service parameter is required"}'
  exit 1
fi

systemctl restart "$SERVICE"
STATUS=$(systemctl is-active "$SERVICE")

if [ "$STATUS" = "active" ]; then
  echo "{\"status\": \"success\", \"service\": \"${SERVICE}\", \"state\": \"${STATUS}\"}"
else
  echo "{\"status\": \"failure\", \"service\": \"${SERVICE}\", \"state\": \"${STATUS}\"}"
  exit 1
fi
```

Bolt Architecture



Bolt Inventory and Plans

```yaml
# inventory.yaml
targets:
  - name: webservers
    targets:
      - web01.example.com
      - web02.example.com
      - web03.example.com
    config:
      transport: ssh
      ssh:
        user: puppet
        private-key: ~/.ssh/puppet_deploy_key
        host-key-check: true

  - name: dbservers
    targets:
      - db01.example.com
    config:
      transport: ssh
      ssh:
        user: puppet
        private-key: ~/.ssh/puppet_deploy_key
```

```
# plans/rolling_restart.pp
plan mymodule::rolling_restart (
  TargetSpec $targets,
  String[1]  $service,
  Integer[1] $batch_size = 1,
) {
  $target_list = get_targets($targets)

  $target_list.each |$target| {
    run_task('mymodule::restart_service', $target, { service => $service })
    ctrl::sleep(5)   # wait before proceeding to next target
  }
}
```

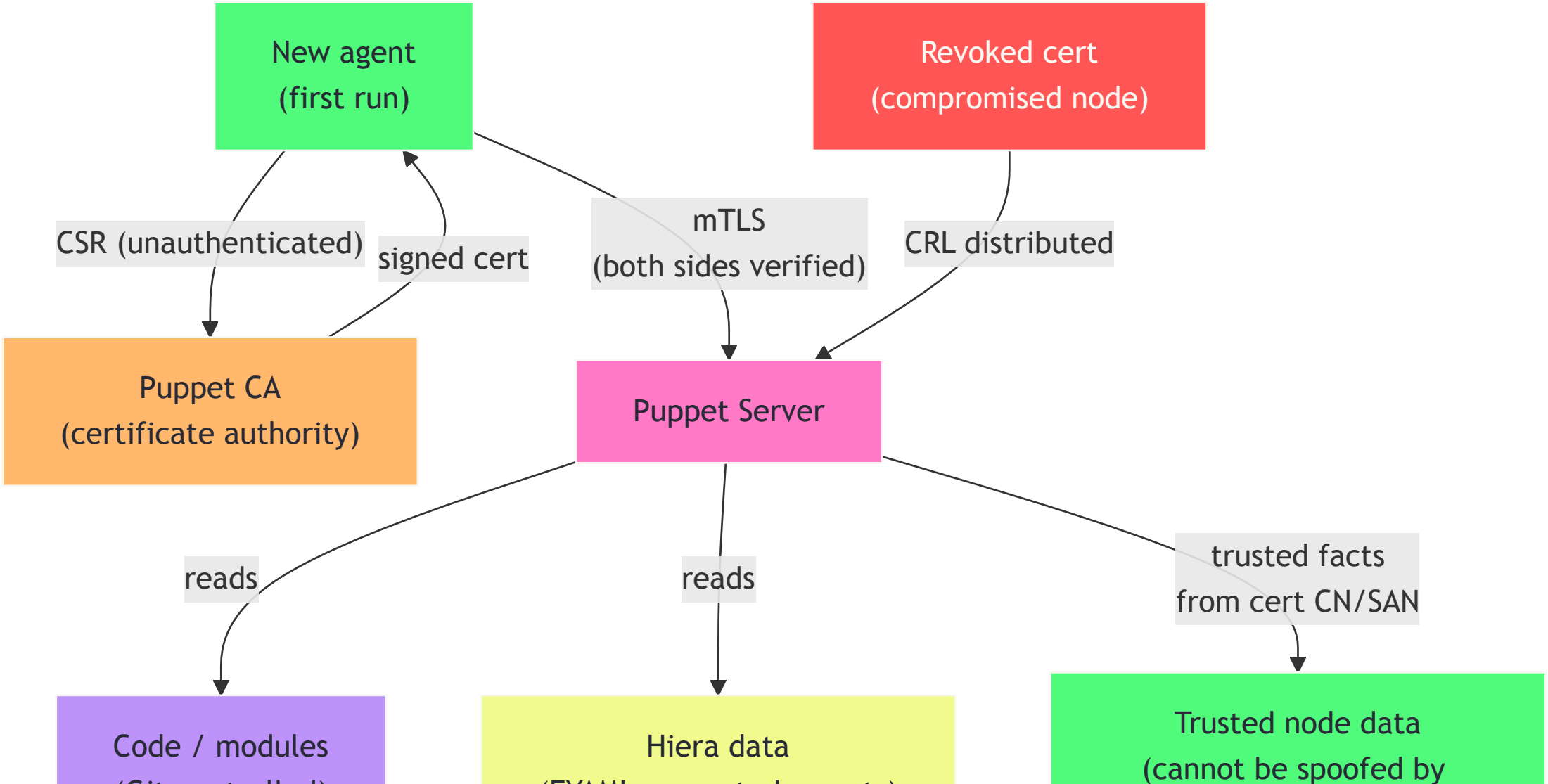
Bolt Plans — Error Handling and Result Sets

Tasks return `ResultSet` objects. Always handle failures explicitly:

```
plan mymodule::deploy (
  TargetSpec $targets,
  String[1]  $version,
) {
  # Run task — returns a ResultSet
  $result = run_task('mymodule::deploy_app', $targets,
    version => $version,
    _catch_errors => true,   # don't abort on first failure
  )

  # Inspect results
  $failed  = $result.error_set.targets
  $success = $result.ok_set.targets

  if $failed.length > 0 {
    $failed.each |$t| {
      out::message("FAILED on ${t.name}: ${$result.find($t.name).error.message}")
    }
    fail_plan("Deployment failed on ${$failed.length} target(s)")
  }

  out::message("Deployed ${version} successfully to ${$success.length} target(s)")
```

Part 19 — Security in Puppet Environments

The Puppet Security Model



Certificate Management — Security Rules

```
Rules for production certificate management:
  ✗ NEVER enable naïve autosigning ($autosign = true) in production
  ✓ Use policy-based autosigning with a challenge password
  ✓ Sign certificates manually for sensitive/privileged nodes
  ✓ Revoke certificates for decommissioned or compromised nodes immediately
  ✓ Monitor the CSR queue — unexpected CSRs are a security signal
  ✓ Set short certificate lifetimes (default is 5 years — consider 1 year)
  ✓ Run the CA on a dedicated, isolated host
  ✓ Backup the CA private key in a hardware security module or vault

# Safe policy-based autosigning
# /etc/puppetlabs/puppet/autosign.rb
#!/usr/bin/env ruby
csr = Puppet::SSL::CertificateRequest.from_s(STDIN.read)
challenge = csr.request_extensions.find { |e| e['oid'] == '1.3.6.1.4.1.34380.1.1.4' }
expected  = File.read('/etc/puppetlabs/puppet/autosign_secret').strip

exit(challenge && challenge['value'] == expected ? 0 : 1)
```

Avoiding Command Injection in Puppet Code

```
# UNSAFE — $user_input could be "alice; rm -rf /"
exec { 'create-user':
  command => "/usr/local/bin/create-user ${user_input}",   # injection risk
}

# SAFE — validate and use shell escaping
$safe_username = shell_escape($username)   # Puppet stdlib function
exec { 'create-user':
  command => "/usr/local/bin/create-user ${safe_username}",
  onlyif  => "id ${safe_username}",
}

# SAFER — use a proper resource type instead of exec
user { $username:
  ensure => present,
}

# UNSAFE — template with unescaped user data
file { '/etc/app/config.conf':
  content => "admin_user = ${facts['external_username']}\n",  # fact injection
}

# SAFE — validate fact values with a Pattern type before use
class myapp (
  Pattern[/\A[a-z0-9_]{1,32}\z/] $admin_user,
) {
  file { '/etc/app/config.conf':
    content => "admin_user = ${admin_user}\n",
  }
}
```

Supply Chain Security for Puppet Modules

```
# Puppetfile — secure practices
# Pin ALL Forge modules to an exact version — never use floating ranges alone
mod 'puppetlabs-apache',  '12.1.0'    # SECURE — exact pin
mod 'puppetlabs-apache',  '>= 12.0.0' # RISKY — could pull unexpected update

# For internal modules from Git — pin to a tag, not a branch
mod 'myorg-webapp',
  :git => 'git@gitlab.example.com:puppet/puppet-webapp.git',
  :tag => 'v3.2.1'    # SECURE

mod 'myorg-webapp',
  :git    => 'git@gitlab.example.com:puppet/puppet-webapp.git',
  :branch => 'main'   # RISKY — any commit to main is immediately deployed
```

Before adding any Forge module:

1. Review the module's source code on GitHub
2. Check the author's track record (puppetlabs-* modules are maintained by Puppet)
3. Check for open security issues in the Forge listing
4. Review what `exec` resources it uses and whether they are guarded
5. Check for hardcoded secrets or world-readable file modes
6. Pin to the reviewed version before deploying to production

Secrets Rotation with EYAML

```
# Step 1 — Generate new key pair
eyaml createkeys --pkcs7-private-key /tmp/new_private_key.pkcs7.pem \
                 --pkcs7-public-key  /tmp/new_public_key.pkcs7.pem

# Step 2 — Re-encrypt all EYAML files with new keys
find data/ -name '*.eyaml' -exec eyaml recrypt \
  --pkcs7-private-key /etc/puppet/keys/private_key.pkcs7.pem \
  --new-pkcs7-private-key /tmp/new_private_key.pkcs7.pem \
  --new-pkcs7-public-key  /tmp/new_public_key.pkcs7.pem \
  {} \;

# Step 3 — Commit re-encrypted data files to Git
git add data/
git commit -m 'chore: rotate EYAML encryption keys'
git push

# Step 4 — Deploy new private key to Puppet Server ONLY
scp /tmp/new_private_key.pkcs7.pem puppet.example.com:/etc/puppetlabs/puppet/keys/
ssh puppet.example.com 'chmod 640 /etc/puppetlabs/puppet/keys/private_key.pkcs7.pem'

# Step 5 — Verify decryption works
puppet lookup myapp::db_password --node test-node.example.com --environment production

# Step 6 — Remove old private key backup
shred -u /tmp/new_private_key.pkcs7.pem
```

Trusted Facts — Securing Classification and Exported Data

Trusted facts come from the agent's **SSL certificate** — they cannot be forged by a malicious agent:

```
# Access trusted facts in manifests
$trusted['certname']                # the node's CN — guaranteed authentic
$trusted['extensions']['pp_role'] # custom OID extension baked into the cert
$trusted['domain']                  # domain portion of certname
```

Custom OID extensions let you embed metadata at cert-signing time:

```
# /etc/puppetlabs/puppet/csr_attributes.yaml  (on the agent before first run)
extension_requests:
  pp_role:        'webserver'
  pp_datacenter:  'ams01'
  pp_environment: 'production'
  pp_application: 'ecommerce'
```

```
# In site.pp — classify nodes using only trusted data
node default {
  $role = $trusted['extensions']['pp_role']
  include "role::${role}"
}

# In exported resources — restrict collection to same datacenter
@@concat::fragment { "backend_${trusted['certname']}":
  target  => '/etc/haproxy/haproxy.cfg',
  content => "  server ${trusted['certname']} ${facts['networking']['ip']}:8080\n",
  tag     => ["haproxy-backend", $trusted['extensions']['pp_datacenter']],
}
```

> **Never classify nodes based on untrusted facts** (`$facts['role']` etc.). Any agent can forge its own facts. Only `$trusted` is safe.

Part 20 — Performance Optimization and Scaling

Finding the Bottleneck

```
Is the agent run slow?

  measure: time puppet agent --test

  → Slow BEFORE "Applying configuration":
      bottleneck is catalog compilation (server-side)
      → profile: puppet catalog compile node --environment ...
      → look for: large exported resource collections, expensive functions,
                  deep Hiera hierarchies with many levels

  → Slow AFTER "Applying configuration":
      bottleneck is resource application (agent-side)
      → look for: slow exec resources, package manager operations,
                  large file resources, slow service start times,
                  large number of resources in catalog

  → Slow PuppetDB submission:
      last step in log: "Submitted catalog"
      → check PuppetDB command queue depth
      → check PostgreSQL performance
```

Puppet Code Performance Anti-Patterns

```
# SLOW — exported resource collection runs a PuppetDB query
# every time this class is compiled
Concat::Fragment <<| tag == $datacenter |>>  # PDB query
Nagios_service   <<| host_name == $certname |>>  # PDB query

# Prefer static Hiera data when possible
$backends = lookup('haproxy::backends', {merge => hash})
$backends.each |$name, $config| {
  concat::fragment { "backend_${name}": ... }
}

# SLOW — large fact collection with expensive system calls
Facter.add('all_installed_packages') do
  setcode { Facter::Core::Execution.execute('rpm -qa --queryformat "%{NAME}\n"') }
end
# This runs rpm -qa on EVERY agent run! Avoid unless necessary.

# SLOW — complex logic in templates
# Bad: EPP/ERB with nested loops and database-style lookups
# Better: pre-process data in the manifest using Ruby functions,
#         pass clean data to the template

# SLOW — large number of file resources without purge scoping
file { $thousands_of_config_files: ensure => file, ... }
# Better: use concat or file_line for incremental changes
```

JRuby and Puppet Server Tuning

```
# /etc/puppetlabs/puppetserver/conf.d/puppetserver.conf
jruby-puppet {
  # Increase pool for more concurrent catalog compilations
  max-active-instances: 6    # rule: min(CPU_cores - 1, 12)

  # Reduce memory leaks from long-running JRuby instances
  max-requests-per-instance: 100000

  # Enable JIT compilation for hot code paths
  compile-mode: jit   # options: off, jit, force
}
```

```
# /etc/default/puppetserver — JVM heap sizing
# Total heap = max-active-instances × ~600 MB
# 6 instances × 600 MB = 3.6 GB → set Xmx4g

JAVA_ARGS="-Xms4g -Xmx4g \
  -XX:+UseG1GC \
  -XX:G1HeapRegionSize=16m \
  -XX:+ParallelRefProcEnabled \
  -XX:MaxGCPauseMillis=200 \
  -Djruby.logger.class=com.puppetlabs.jruby_utils.jruby.Slf4jLogger"
```

Agent Run Optimization

```
# puppet.conf — agent tuning
[agent]
runinterval = 1800        # 30 min is the standard; don't go shorter without scaling
splay = true              # stagger agent runs to avoid thundering herd
splaylimit = 1800         # max random delay = one full runinterval
usecacheonfailure = true  # apply last known-good catalog if server unreachable
noop = false              # set true to make an environment read-only by default
```

Reducing catalog application time:

```
# Use File resources with source from fileserver — Puppet handles delta sync
file { '/etc/myapp/config.tar.gz':
  source   => 'puppet:///modules/myapp/config.tar.gz',
  checksum => sha256,   # only download if checksum differs
}

# Set file resources to noop in sensitive environments
file { '/etc/critical-config':
  ensure => file,
  noop   => true,       # Puppet will report drift but not apply changes
}
```

Profiling Catalog Compile Time

When "Applying configuration" starts late, the bottleneck is server-side. Pinpoint it:

```
# 1. Compile a catalog for a specific node and time it
time puppet catalog compile web01.example.com \
  --environment production \
  --terminus rest 2>&1 | tail -5

# 2. Enable detailed compile timing in Puppet Server logs
# /etc/puppetlabs/puppetserver/conf.d/puppetserver.conf
profiler: { enabled: true }

# 3. Read compile timing from Puppet Server log
grep "Compiled catalog for" /var/log/puppetlabs/puppetserver/puppetserver.log \
  | awk '{print $NF, $(NF-2)}' | sort -rn | head -20

# 4. Check PuppetDB for compile times across all nodes (PQL)
puppet query 'reports[certname, metrics] {
  latest = true and metrics.time.total > 30
  order by metrics.time.total desc
  limit 10
}'
```

Part 21 — Foreman as Lifecycle Management and ENC Tool

Foreman Overview

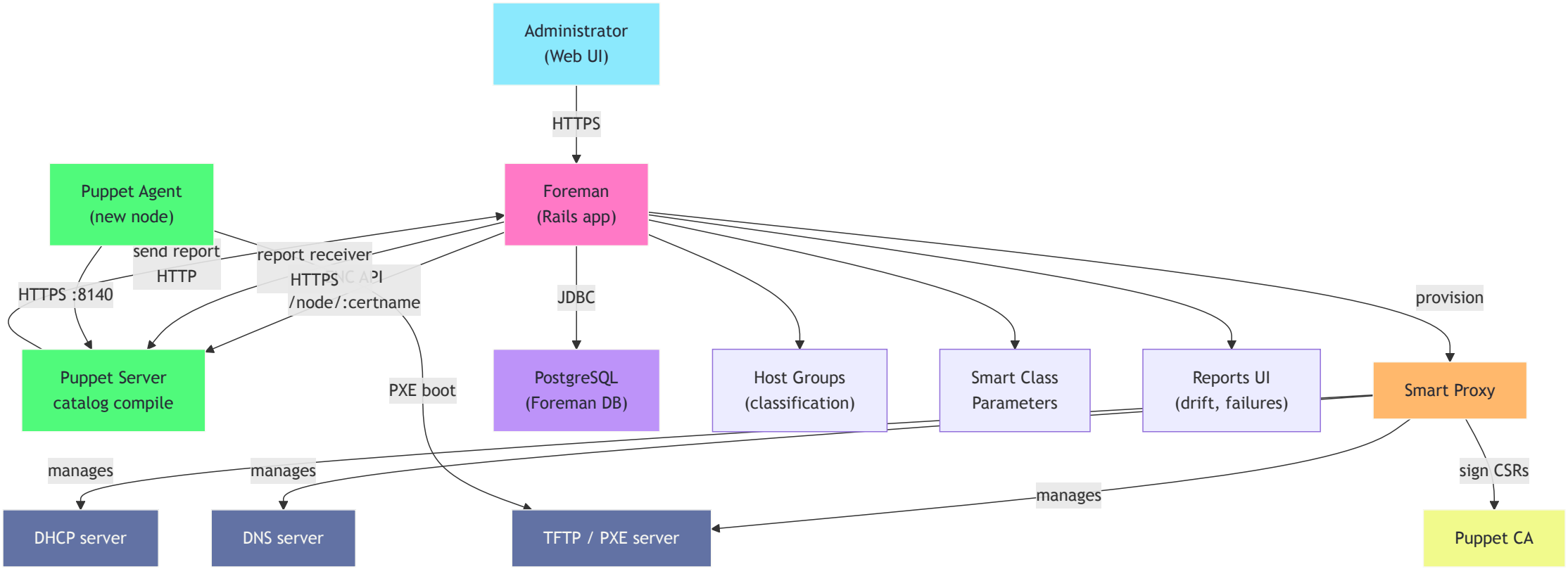
Foreman is an open-source lifecycle management tool that integrates tightly with Puppet Open Source:

Foreman capability	Description
Host inventory	CMDB-style view of all nodes with facts and reports
External Node Classifier	Assigns Puppet classes and parameters to nodes via GUI
Reporting	Receives Puppet agent reports; visualises drift and failures
Provisioning	PXE-based bare-metal provisioning; cloud provisioning

Foreman capability	Description
DHCP / DNS / TFTP management	Lifecycle from first boot to fully managed
Smart Proxy	Remote component that manages local services on behalf of Foreman
Role-Based Access Control	Multi-team, multi-tenant access to the Puppet infrastructure

> Foreman fills the operational web UI gap in Puppet Open Source. Puppet Enterprise provides similar capabilities natively.

Foreman Architecture



Foreman as External Node Classifier

When configured as ENC, Foreman responds to Puppet Server's ENC call:

```
Puppet Server calls:
  GET https://foreman.example.com/node/<certname>?format=yaml

Foreman returns YAML like:
  ---
  classes:
    role::webserver: {}
    profile::base::ntp:
      servers:
        - 0.pool.ntp.org
        - 1.pool.ntp.org
    profile::monitoring:
      prometheus_port: 9100
  environment: production
  parameters:
    datacenter: ams01
    team: platform
```

Foreman Host Groups and Classification

```
Foreman classification hierarchy:
  Host Group: "Europe / AMS01 / Webservers"
    └─ parent: "Europe / AMS01"
         └─ parent: "Europe"
              └─ parent: "All Hosts"

  Each level can inherit and override:
    All Hosts:
      - profile::base (NTP, SSH hardening, monitoring agent)

    Europe:
      - profile::base::timezone  [timezone = Europe/Amsterdam]

    Europe / AMS01:
      - profile::base::ntp  [servers = [ntp1.ams01.example.com]]

    Europe / AMS01 / Webservers:
      - role::webserver
      - profile::web::nginx  [worker_processes = 8]
```

Smart Class Parameters allow overriding individual class parameters at any level of the host group hierarchy without modifying Hiera data — useful for teams that need GUI-based overrides.

Foreman Report Analysis

```
# Foreman receives reports from Puppet agents automatically
# Configure in puppet.conf on agents:
[main]
report = true
reports = foreman    # comma-separated; can use foreman,puppetdb

# Foreman REST API — programmatic report access
curl -u admin:password https://foreman.example.com/api/v2/reports \
  --header "Accept: application/json" \
  | python3 -m json.tool

# Find all failed hosts
curl -u admin:password \
  "https://foreman.example.com/api/v2/hosts?search=last_report_status+%3D+failed" \
  --header "Accept: application/json"

# Trigger a Puppet run from Foreman (requires Smart Proxy with Puppet feature)
curl -u admin:password \
  -X POST "https://foreman.example.com/api/v2/hosts/web01.example.com/puppetrun" \
  --header "Accept: application/json"
```

Foreman Provisioning Concepts

```
Bare-metal provisioning flow with Foreman:

  1. New physical server powers on
  2. PXE boot → DHCP lease from Smart Proxy → TFTP: pxelinux.0
  3. Foreman identifies MAC address, assigns Host record
  4. PXE loads kickstart/preseed template from Foreman
  5. OS install completes (kickstart snippet runs puppet-agent install)
  6. Puppet agent starts, submits CSR to CA
  7. Foreman Smart Proxy auto-signs (policy-based) or operator signs
  8. First Puppet run classifies node via ENC (Foreman host group)
  9. Puppet applies full configuration
  10. Foreman marks host as 'built' — provisioning complete

Foreman templates:
  - OS installation templates (kickstart, preseed, AutoYaST)
  - PXE templates (pxelinux, iPXE, Grub2)
  - Finish scripts (run post-install actions)
  - All templates are Embedded Ruby (ERB) — can access Foreman host attributes
```

Part 22 — Enterprise Puppet Best Practices

Architecture Best Practices

```
✓ Start simple — single server + PuppetDB is right for < 200 nodes
✓ Add compile servers only when catalog compile time is the bottleneck
✓ Separate the CA from compile servers from day one — retrofitting is hard
✓ Use a control repository from the very first line of Puppet code
✓ Use r10k for ALL code deployment — never edit code directly on the server
✓ Monitor all Puppet components: server, PuppetDB, agent runs, reports
✓ Set up alerting for: failed agent runs, stale nodes, PuppetDB queue backlog
✓ Document your infrastructure: architecture diagram + operational runbook
✓ Test disaster recovery: can you rebuild the Puppet Server from scratch?
```

Code Best Practices

```
✓ Role/profile pattern — always; no exceptions for nodes with > 2 classes
✓ Typed parameters on ALL public class definitions
✓ Hiera for ALL configuration data — no hardcoded values in manifests
✓ Module-level Hiera data for default values — not in init.pp defaults
✓ rspec-puppet tests for all critical logic paths and type validations
✓ Semantic versioning for internal modules (MAJOR.MINOR.PATCH)
✓ Pin all module versions in Puppetfile — never use floating versions in production
✓ puppet-lint in CI — zero warnings policy
✓ puppet parser validate in CI — run on every commit
✓ Code review for all changes to production code
✗ Don't use node-level manifests in site.pp — use ENC or role classification
✗ Don't use class inheritance (except from stdlib) — use parameters instead
✗ Don't use params.pp pattern — use Hiera APL instead
✗ Don't put secrets in Git — ever (use EYAML or Vault)
```

Operational Best Practices

Practice	Why
Staged deployments	Deploy to dev → staging → production; never skip stages
Change windows for production	Coordinate Puppet code changes with change management
Noop runs before production apply	`puppet agent --test --noop` across production before enabling
Report review	Check PuppetDB / Foreman reports after every production deployment

Practice	Why
Certificate lifecycle management	Track expiry dates; automate renewal alerts 60 days before
Stale node cleanup	Nodes inactive > 30 days: deactivate in PuppetDB, revoke cert
Regular backup verification	Test restore of Puppet Server + PuppetDB + CA quarterly
Module security review	Review all Forge modules before first use; re-review on upgrade

Organizational Best Practices

```
Module ownership:
  Each module has a designated maintainer (team or individual)
  Changes require review from the module owner
  No "orphan" modules without a responsible owner

Review process:
  All changes go through merge requests
  At least one reviewer before merge to main/production
  Reviewer checks: correctness, types, tests, documentation, security

Documentation standards:
  Each module: README with usage examples, parameter reference, limitations
  Control repository: architecture overview, Hiera hierarchy explanation
  Operations: runbooks for common tasks (add node, debug failure, rotate secrets)

Training:
  New team members complete basic + expert Puppet training before writing production code
  Regular knowledge-sharing sessions (bi-weekly or monthly)
  Maintain internal best-practice guide and code examples
```

Day 3 — Summary

Extension development mastery:

- Custom Ruby facts with structured output and security considerations
- Custom Puppet functions with typed dispatch and tests
- Custom types and providers with idempotent prefetch/flush patterns

Security mastery:

- Trust model: facts vs. trusted facts
- EYAML key management and rotation
- Code security: command injection, supply chain, secret management

Operations mastery:

- Performance profiling: server, code, agent, PuppetDB
- JVM and JRuby tuning for production scale
- Foreman as a GUI layer for ENC, provisioning, and reporting

Enterprise mastery:

- Architecture, code, operational, and organizational best practices
- The complete picture of a production-grade Puppet installation

Day 3 — Practical Exercises

Exercise 1: Custom Facts, Functions, and a Complete Type + Provider

Exercise 2: Foreman Classification and Puppet Security Review

Seminar Complete

Congratulations. You have covered the full expert landscape of Puppet infrastructure design, advanced language features, professional module development, testing, Ruby-based extensions, security, performance, and lifecycle management with Foreman.

Next steps:

- Apply the role/profile pattern and control repository to your environment
- Add rspec-puppet tests to your most critical modules
- Set up monitoring and alerting for your Puppet infrastructure
- Review your security posture: certificate policy, Hiera EYAML, module supply chain
- Document your architecture and operational runbooks